

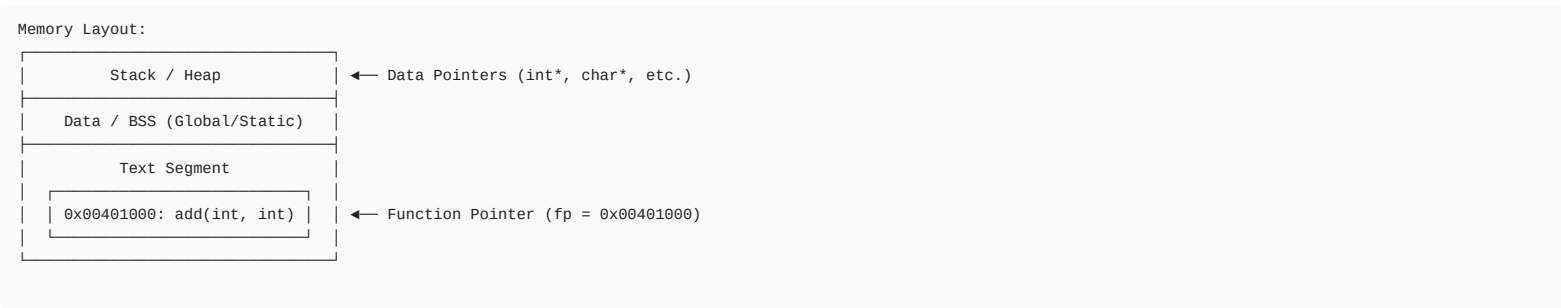
Topic 5: Pointers to Functions (Function Pointers)

1. High-Yield Conceptual Theory

1.1 Memory Model & Core Definition

In C, executable code resides in the **Text (Code) Segment** of memory. Every function begins at a specific memory entry address.

- A **function pointer** is a pointer variable that stores the entry point address of an executable function, rather than data in the Stack, Heap, or Data segment.
- Function pointers enable **dynamic dispatch**, **callback mechanisms** (e.g., in event-driven systems and sorting algorithms like `qsort`), and **jump/lookup tables**.



1.2 Syntax & The Precedence Distinction

The declaration of a function pointer requires strict attention to parentheses:

```
return_type (*fp_name)(parameter_type_list);
```

The Critical Distinction

- `int *fp(int, int);`
- `()` has higher precedence than unary `*`.
- This declares a **function** named `fp` that takes two `int` parameters and returns a **pointer to an integer** (`int *`).
- `int (*fp)(int, int);`
- The parentheses force `*` to bind to `fp` first.
- This declares a **pointer** `fp` to a function that takes two `int` arguments and returns an **integer** (`int`).

1.3 Assignment and Invocation Mechanics

In C, a function designator automatically decays into a pointer to that function, similar to how an array name decays to a pointer to its first element.

```
int add(int a, int b) { return a + b; }

int main() {
    int (*fp)(int, int);

    // Both assignments are 100% equivalent in standard C:
    fp = add;    // Implicit decay
    fp = &add;   // Explicit address-of operator

    // Both invocations are 100% equivalent in standard C:
    int r1 = fp(3, 4);    // Implicit dereference
    int r2 = (*fp)(3, 4); // Explicit dereference

    return 0;
}
```

1.4 Complex Declarations: The Right-Left (Clockwise/Spiral) Rule

To interpret any complex C declaration:

1. Start at the identifier.
2. Look to the **right** (for `()` or `[]`).
3. Look to the **left** (for `*`).
4. Step outside the current pair of parentheses and repeat right-then-left.

Common GATE Declaration Patterns

- `int (*arr[10])(float);`
1. `arr` is an **array of 10...**

2. * → pointers to...
3. (float) → functions taking 'float'...
4. int → and returning 'int'.

- int *(*fp)(char);
- fp is a pointer to a function taking char and returning a pointer to int (int *).
- int *(*fp)(int))(void);

1. fp is a pointer to a function taking int ...
2. returning a pointer to a function...
3. taking void ...
4. and returning int .

1.5 Simplifying with typedef

Complex function pointer signatures become clean and readable using typedef :

```
// Defines a type 'Operation' as a pointer to a function taking two ints and returning an int
typedef int (*Operation)(int, int);

Operation op = add; // Clean syntax
int result = op(10, 20);

// Array of function pointers:
Operation table[4] = { add, sub, mul, div };
```

1.6 Function Pointers as Callbacks: qsort

A prime practical application tested in GATE is standard library callbacks, such as qsort in <stdlib.h> :

```
void qsort(void *base, size_t nmemb, size_t size,
           int (*compar)(const void *, const void *));
```

The comparator function compar(a, b) must return:

- < 0 if the element pointed to by a should precede b (ascending order).
- 0 if *a is equivalent to *b .
- > 0 if *a should follow *b .

```
int cmp_asc(const void *a, const void *b) {
    return (*(int *)a - *(int *)b); // Caveat: subtraction can cause integer underflow
}
```

2. Common GATE Pitfalls & Edge Cases

1. Arithmetic on Function Pointers is Strictly Forbidden

Because functions have no quantifiable sizeof (they represent code, not a data object with a runtime size), **pointer arithmetic on function pointers is an error under ISO C**:

```
fp++; // COMPILER ERROR: cannot increment a pointer to function
fp = fp + 1; // COMPILER ERROR: invalid operands to binary +
```

(Note: GCC provides an extension where sizeof(function) == 1, but in standard C and GATE, function pointer arithmetic is strictly invalid).

2. Equality vs. Relational Comparisons

- **Valid**: Comparing a function pointer to NULL or testing equality with another function pointer (fp == add or fp1 != fp2).
- **Invalid/Undefined**: Ordering comparisons (<, <=, >, >=) between function pointers are undefined in C.

3. Signature Mismatches

C allows casting between different function pointer types, but calling a function through a pointer whose type does not match the actual function definition leads to **Undefined Behavior (UB)**:

```
void target(int a);
void (*fp)(void) = (void (*)(void))target;
fp(); // UNDEFINED BEHAVIOR: Argument mismatch during invocation
```

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2007)

Question:

Which of the following explains the declaration:

```
int (*f)(int*)
```

- (A) f is a function that takes an integer pointer as argument and returns an integer
- (B) f is a function that takes an integer as argument and returns an integer pointer
- (C) f is a pointer to a function that takes an integer pointer as argument and returns an integer
- (D) f is a pointer to a function that takes an integer as argument and returns an integer pointer

Step-by-step Solution:

1. Identify the core identifier: f .
2. Parentheses $(*f)$ indicate $*$ binds to f first $\rightarrow$ **f is a pointer.**
3. Outside parentheses to the right: $(int*) \rightarrow$ **to a function taking an integer pointer ($int*$) as argument.**
4. To the left: $int \rightarrow$ **and returning an integer (int).**
5. Combining these parts: f is a pointer to a function that takes an integer pointer as argument and returns an integer.

Correct Option: (C)

PYQ 2 (GATE CS 2014)**Question:**

Consider the following C program:

```
#include <stdio.h>

int f1(int (*f)(int)) {
    int n = 5;
    return (*f)(n);
}

int f2(int n) {
    return n * n;
}

int main() {
    printf("%d\n", f1(f2));
    return 0;
}
```

What is the output printed by the program?

Step-by-step Solution:

1. $f1$ is a function that takes a function pointer f with signature $int (*) (int)$ as a parameter.
2. $f2$ is a function with signature $int (int)$: returns n^2 .
3. In $main()$, $f1(f2)$ is called:
 - The function name $f2$ decays to a pointer to $f2$.
 - Parameter f in $f1$ now points to $f2$.
4. Inside $f1$:
 - $n = 5$.
 - $(*f)(n)$ invokes $f2(5)$.
 - $f2(5)$ computes $5 \times 5 = 25$.
 - $f1$ returns 25.
5. $printf$ outputs 25.

Output: 25

PYQ 3 (GATE CS 2008 — Tracing with Function Pointer Arrays)**Question:**

Consider the following C code segment:

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int main() {
    int (*ops[3])(int, int) = { add, sub, mul };
    int val = 10;
    for (int i = 0; i < 3; i++) {
```

```
        val = ops[i](val, i + 1);
    }
    printf("%d", val);
    return 0;
}
```

What will be printed by the program?

Step-by-step Solution:

- ops is an array of 3 function pointers:
 - ops[0] = add
 - ops = sub
 - ops = mul
- Initial value: val = 10 .
- Iteration i = 0:
 - Calls ops[0](val, 0 + 1) → add(10, 1) = 11 .
 - val = 11 .
- Iteration i = 1:
 - Calls ops(val, 1 + 1) → sub(11, 2) = 9 .
 - val = 9 .
- Iteration i = 2:
 - Calls ops(val, 2 + 1) → mul(9, 3) = 27 .
 - val = 27 .
- Final output printed: 27 .

Output: 27

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Right-Left Rule on Deeply Nested Declarations

Question:

Decode the following declaration in plain English:

```
void (*signal(int sig, void (*handler)(int)))(int);
```

Detailed Explanation:

Apply the Right-Left Rule systematically:

- Start at identifier signal .
- Look to the right: (int sig, void (*handler)(int))
 - signal is a **function** taking two parameters:
 - an int sig
 - and a function pointer handler (which takes an int and returns void) .
- Look to the left of signal: *
 - signal returns a **pointer**...
- Step outside parentheses to the right: (int)
 - ...to a **function** taking an int ...
- Look to the far left: void
 - ...and returning void .

Result: signal is a function taking an integer and a function pointer, returning a pointer to a function that takes an integer and returns void . (This is the exact signature of the POSIX signal function).

Practice Question 2: Function Pointer Redundant Operators

Question:

Consider the following program:

```
#include <stdio.h>

void show() {
    printf("GATE");
}
```

```

int main() {
    void (*p)() = show;
    (*****p)();
    (*****show)();
    return 0;
}

```

Does this code compile, and what does it print?

Detailed Explanation:

1. In C, whenever a function designator appears, it automatically converts to a pointer to that function.
2. When you dereference a function pointer `*p`, it yields the function designator again.
3. That function designator immediately converts back to a pointer to the function.
4. Hence, applying any number of `*` (dereference) operators to a function or function pointer evaluates to the same function pointer:

$$p \equiv *p \equiv **p \equiv ***p$$

5. Both `(*****p)()` and `(*****show)()` are valid invocations of `show()`.
6. Each prints "GATE".

Output: GATEGATE

Practice Question 3: Illegal Arithmetic on Function Pointers

Question:

Consider the following snippet:

```

int foo(int x) { return x + 1; }
int bar(int x) { return x + 2; }

int main() {
    int (*fp)(int) = foo;
    fp = fp + 1;
    printf("%d", fp(5));
    return 0;
}

```

What is the outcome of compiling and running this code under standard ISO C?

Detailed Explanation:

1. `fp` is of type `int (*)(int)`.
2. In ISO C (C89, C99, C11, C17), pointer arithmetic is only defined for pointers to complete **object** types.
3. Functions are not objects; a function type has no complete object size.
4. Therefore, `fp + 1` violates the C standard constraint on additive operators.
5. Standard C compilers issue a compile error or strict diagnostic reject.

Result: Compilation Error (Invalid operands / Arithmetic on pointer to function)

5. Quick Revision & Edge-Case Checklist

- ☐ `int *f(void)` returns a pointer to `int`. `int (*f)(void)` is a pointer to a function returning `int`.
- ☐ Function names decay to pointers automatically: `fp = func;` and `fp = &func;` are identical.
- ☐ Invocation is symmetric: `fp()` and `(*fp)()` are identical.
- ☐ Multiple dereferences `(***func)()` are valid and resolve to the function call.
- ☐ Pointer arithmetic (`fp++`, `fp + n`, `fp1 - fp2`) is **forbidden** on function pointers in standard C.
- ☐ Equality checks (`fp == NULL`, `fp1 == fp2`) are valid; relational ordering (`<`, `>`) is undefined.
- ☐ Jump tables/dispatch tables use arrays of function pointers: `int (*dispatch[N])(params)`.
- ☐ `typedef` makes nested function pointer signatures clean and less error-prone.

This concludes **Topic 5: Pointers to Function**. Reply with **"Continue"** to proceed to the next topic: **Functions Returning Pointers**.